

# Affordmed Round 1 Assessment

*What you will actually receive in the interview process — decoded stage by stage, with what to actually prepare*

---

## Why This Document Exists

Affordmed's Round 1 is not a typical multiple-choice coding test. It's a 3-hour, project-style assessment where you incrementally build a real backend (and, for some tracks, a frontend) against a running storyline — a campus placement notification system. The stages are designed to feel like a real sprint at a real company: requirements evolve, previous work has to be revised, and every decision you make can be questioned afterwards. This document breaks down every stage in plain language, explains *why* each one exists, and tells you exactly what to revise before you sit for it.

## The Big Picture

There are two tracks, and you will be told which one applies to you before the test starts:

- **Full Stack Track:** Pre-Test (Logging Middleware) + 7 stages, ending with a React/Next frontend. Time allotted for the coding-heavy parts is roughly double that of the other track.
- **Backend-Only Track:** Pre-Test (Logging Middleware) + a standalone algorithm problem (Vehicle Maintenance Scheduler) + the same Campus Notifications stages, minus the frontend stage.

Both tracks share the same spine: build something small, then keep extending it as new constraints appear. This is deliberate. The assessment is not really testing “can you write code” — you already proved that when you cleared the aptitude/coding round to get here. It is testing **how you think as requirements change**, which is exactly what a junior engineer does in their first six months on a real team.

## Ground Rules That Apply Throughout

- **Time limit:** 3 hours total, with no extra time granted for pushing your work to GitHub. Budget the last 10–15 minutes purely for committing and pushing.
- **Mandatory logging:** every piece of code you write must call the Logging Middleware you build in the Pre-Test stage. Built-in language loggers or `console.log` are explicitly not allowed — examiners will check for this.
- **No auth required:** assume all users and APIs are pre-authorised. You do not need to build login, registration, or session handling — building this anyway wastes time you don't have.
- **Commit often:** a single end-of-test commit scores *lower* than frequent, milestone-based commits, even if the final code is identical. Commit after every stage, minimum.
- **Stages build on each other:** you're allowed to revise earlier stages as later ones change your thinking — and you're expected to. Submissions are graded both stage-by-stage and as a whole.

## Pre-Test: The Logging Middleware

Every candidate, on every track, builds this first — before touching any of the real problem stages. You write one reusable function:

```
Log(stack, level, package, message)
```

...that calls a test-server API every single time it's invoked. You then have to sprinkle calls to this function throughout everything else you build for the rest of the test.

### What Each Field Actually Means

- **stack** — was this a “frontend” or “backend” event?
- **level** — how serious is it: debug, info, warn, error, or fatal?
- **package** — which part of your code raised it (e.g. controller, db, cache, cron\_job)?
- **message** — a specific, human-readable description — not a generic “something went wrong.” Compare: “error” vs. “received string, expected bool for field 'isRead'”. The second one is what a real engineer would want to see at 2 a.m. while debugging a production incident.

### What It's Really Testing

- Can you build a small, reusable *package* — not just a one-off script?
- Do you understand structured logging as a concept, not just as a print statement?
- Did you make logging **fire-and-forget** (non-blocking)? In plain terms: if the logging server is slow or down, your actual application should keep working — a log call should never be allowed to crash or freeze the real feature around it.

**Expect to be asked:** “Why did you choose these specific fields?” and “What happens to your app if the log server is unreachable — does it hang, retry, or silently drop the log?” Have a real answer, not “I didn't think about that.”

## Backend-Only Track: Vehicle Maintenance Scheduler

This problem is dressed up as a logistics scenario, but underneath it is a classic algorithm you've likely already studied: the **0/1 Knapsack problem**.

- Each repair task has a **Duration** (how many mechanic-hours it takes) — this plays the role of *weight*.
- Each task has an **Impact** score (how important it is) — this plays the role of *value*.
- Each depot has a fixed **MechanicHours** budget — this plays the role of *capacity*.
- The goal: pick the subset of tasks that maximises total Impact without exceeding the MechanicHours budget — exactly the Knapsack objective.

**In plain terms:** you cannot just sort tasks by “highest impact first” and keep adding them until you run out of hours. That greedy approach can lock you into a bad combination early and miss a better one later — the classic Knapsack trap. The intended solution uses **dynamic programming**: build a table of “best possible impact using the first *i* tasks and *h* hours,” and grow it up from small subproblems.

**Expect to be asked:** “Why doesn't greedy-by-impact work here?” and “What's the time and space complexity of the solution you actually submitted?” Be ready to trace through a tiny 3-4 task example by hand — examiners often ask you to do this live on a whiteboard or shared screen.

## The Main Challenge: Campus Notifications

This is the heart of the assessment for both tracks. You are a backend developer (or full-stack developer) on a campus notification platform where students get real-time updates about Placements, Events, and Results. It runs as **one continuous story across several stages** — each one deliberately breaks or extends what you built in the one before it.

| Stage | What You Actually Do                                                                                                                                                    | The Concept Being Tested                                                                                                                                             |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Design REST API endpoints, JSON request/response contracts, and a real-time notification mechanism (WebSocket, Server-Sent Events, or polling).                         | API contract design. Can you think through endpoints and data shapes <i>before</i> writing implementation code — a senior-level habit for a fresher to already have. |
| 2     | Choose SQL or NoSQL for persistent storage, write the schema, and justify the choice.                                                                                   | Data modeling. Do you pick a database because of the shape of the data and its access patterns — not just because it's what you know?                                |
| 3     | A given query is slow at 5,000,000 rows. Diagnose why, propose an index, and judge whether “index every column” is good advice.                                         | Real index/query understanding — arguably the single best signal in the entire assessment for database maturity.                                                     |
| 4     | The database is overwhelmed because notifications are re-fetched on every single page load. Propose a fix.                                                              | Do you reach for caching (in-memory or Redis) and reduced polling, instead of just “add more servers”?                                                               |
| 5     | A bulk “Notify All” job to 50,000 students partially fails — 200 emails don't send. Redesign the process to be reliable.                                                | Idempotency and reliability. Do you know why a DB write and an external network call (email) shouldn't happen as one fragile, all-or-nothing step?                   |
| 6     | Build a “Priority Inbox” that always shows the top-N most important unread notifications, ranked by type and recency — as real code, not pseudocode.                    | Top-K / heap thinking. Do you reach for a priority queue instead of re-sorting the entire list every time a new notification arrives?                                |
| 7     | (Full Stack track only) Build a responsive React/Next frontend showing all notifications and the Priority Inbox, distinguishing read vs. unread, on desktop and mobile. | Can you turn a backend contract into a genuinely usable UI under real constraints — Material UI only, no component-library shortcuts?                                |

*Note: for the Backend-only track, Stage 7 is dropped entirely — there is no dedicated frontend requirement. Everything else applies to both tracks in the same spirit.*

# How to Actually Prepare for Each Stage

## Stage 1 — API Design

- Practice naming endpoints predictably: GET /notifications, PATCH /notifications/:id/read — not /getNotif or /notif\_update.
- Know the difference between polling, Server-Sent Events, and WebSockets, and be able to say in one sentence why you'd pick one over another for this use case.

## Stage 2 — Database Design

- Be able to justify SQL vs. NoSQL for a specific shape of data — not just recite “SQL is relational, NoSQL is flexible.”
- Practice drawing a simple ER diagram by hand: entities, primary keys, foreign keys, and the cardinality between them (see the SQL module in the main workshop deck).

## Stage 3 — The Slow Query

- Learn to read a query and immediately spot: no index on the filtered/sorted columns, SELECT \* pulling unnecessary columns, and sorting a huge result set.
- Practice writing a **composite index** and explaining why column order in that index matters (the leftmost column should be the one used in equality filters).
- Be ready to explain why indexing every column is bad: each extra index slows down every write and costs storage — indexes are a trade-off, not a free upgrade.

## Stage 4 — Caching & Scale

- Know the difference between an in-memory cache (fast, per-server, lost on restart) and Redis (shared across servers, survives restarts).
- Practice explaining **cache invalidation** in one sentence: how does the cache find out a new notification has arrived, so it doesn't serve stale data?

## Stage 5 — Reliability

- Learn the word **idempotent** cold: an idempotent operation gives the same end result even if it runs twice by accident — crucial for retrying a failed bulk job safely.
- Practice explaining why you'd decouple “save to DB” from “send email” (e.g. using a queue), instead of doing both in one blocking loop.

## Stage 6 — Priority Inbox Algorithm

- Revise **heaps / priority queues** and be able to state the complexity of maintaining a running top-N as new items keep arriving, versus re-sorting the whole list each time.
- Write this from scratch in your strongest language before the test — don't meet a heap implementation for the first time during the assessment.

## Stage 7 — Frontend (Full Stack only)

- Practice building one small Material UI page from scratch beforehand if you haven't used MUI recently — don't let component-library unfamiliarity eat your time budget.
- Plan your read/unread state management (local state vs. a shared store) before you start coding, not while you're halfway through a page.

## Quick Glossary — Terms Worth Knowing Cold

If any of these feel shaky, that's exactly what to revise before the test — every single one appears somewhere in the stages above.

|                                 |                                                                                                                                                                                                                  |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Idempotent</b>               | An operation that produces the same end result no matter how many times it runs. Critical for safely retrying a failed job without duplicating side effects (like sending the same email twice).                 |
| <b>Composite Index</b>          | A single database index built across more than one column (e.g. studentID + isRead). Column order in the index matters — put the column used for equality checks first.                                          |
| <b>Cache Invalidation</b>       | The process of removing or refreshing cached data once the underlying data changes, so users don't see stale information.                                                                                        |
| <b>Heap / Priority Queue</b>    | A data structure that always gives you the highest (or lowest) priority item in $O(\log n)$ time — the standard tool for “top-N” problems where new items keep arriving.                                         |
| <b>0/1 Knapsack</b>             | A classic optimisation problem: given items with a weight and a value, and a maximum total weight, pick the subset of items with the highest total value. Solved with dynamic programming, not greedy selection. |
| <b>Correlated Subquery</b>      | A subquery that references a column from the outer query, so it re-runs once per outer row. Slower than a non-correlated subquery, which runs only once.                                                         |
| <b>ENUM</b>                     | A column type restricted to a fixed set of allowed values (e.g. 'Event' / 'Result' / 'Placement') — used instead of a free-text string to guarantee valid data.                                                  |
| <b>SSE (Server-Sent Events)</b> | A one-way channel where the server pushes updates to the browser over a single long-lived HTTP connection — simpler than WebSockets when the client never needs to send data back in real time.                  |
| <b>Fire-and-Forget</b>          | Calling a function (like a logger) without waiting for its result, so a slow or failing call can't block or crash the main flow of the program.                                                                  |
| <b>MVC</b>                      | Model-View-Controller — a way of separating an app's data (Model), interface (View), and request-handling logic (Controller) into distinct layers.                                                               |

## Common Mistakes Students Make on This Assessment

- **Skipping the Logging Middleware requirement mid-way** — falling back to `console.log` under time pressure. Examiners check for this specifically; it costs real points.
- **Solving Stage 6 or the Knapsack problem with pseudocode only** — both explicitly require working, runnable code plus output screenshots, not a description of the approach.
- **One giant commit at the end** — this alone can lower your score even if the final code is perfect. Commit after every stage.
- **Adding an index on every column “to be safe”** — this is presented in the assessment itself as bad advice, and treating it as correct is a red flag.
- **Ignoring the time budget for pushing to GitHub** — there is no extra time granted for this, so a great solution that never gets pushed scores zero for that stage.

## Why All of This Matters For Your Interview

The technical interview that follows Round 1 is not a generic DSA/SQL grilling session — it is almost entirely a **defense of what you personally submitted**. Interviewers open your GitHub repository and your `notification_system_design.md` file directly and ask you to justify specific choices, stage by stage, in front of them.

### Be Ready to Answer, About Your Own Submission:

- Why did you choose these specific log fields — what would you add with more time?
- Walk me through one REST endpoint end-to-end: request, headers, response, status codes.
- Why was the original query slow, and how exactly does your fix change the computation cost?
- What did you cache, where, and how do you invalidate it the moment new data arrives?
- What makes your bulk-notify redesign idempotent if the same job accidentally runs twice?
- Why a heap over sorting the whole list every time — what's the actual complexity gain?
- Which requirement did you not fully complete, and why? Honesty beats bluffing here.

---

*Treat the interview as Round 1's oral defense, not a brand-new test. You already did the hard thinking under real time pressure — now you just have to explain it out loud, calmly and with the reasoning fresh in your mind. Re-read your own design document the night before, and you'll walk in already ahead of most candidates.*